Building femm-qt on macOS

D. Meeker

September 11, 2026

1 Overview

femm-qt is a cross-platform port of FEMM (Finite Element Method Magnetics) using Qt 6 for the GUI and CMake/Ninja for the build system. This document describes how to build it from source on macOS (Intel or Apple Silicon).

The build produces six executables:

- `femm_gui` — the main GUI application
- `femm_headless` — headless IPC server (no GUI)
- `fkn` — magnetics solver
- `belasolv` — electrostatics solver
- `hsolv` — heat flow solver
- `csolv` — current flow solver

2 Prerequisites

2.1 Xcode Command Line Tools

Install the Apple compiler toolchain (Clang, make, git, etc.) from Terminal:

```
xcode-select --install
```

Follow the prompts to complete the installation. This provides `clang++`, `git`, and standard system headers.

2.2 Homebrew

If not already installed, install the Homebrew package manager:

```
/bin/bash -c "$(curl -fsSL \
    https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Follow the post-install instructions to add Homebrew to your `PATH`.

2.3 CMake and Ninja

```
brew install cmake ninja
```

2.4 Qt 6

Install Qt 6 via Homebrew:

```
brew install qt@6
```

Homebrew installs Qt to a prefix such as `/opt/homebrew/opt/qt@6` (Apple Silicon) or `/usr/local/opt/qt@6` (Intel). Note this path; it is needed for the CMake configure step.

To find your Qt prefix:

```
brew --prefix qt@6
```

3 Getting the Source

If you have a source zip, unzip it:

```
unzip femm-qt-src.zip
cd femm-qt
```

Or, if the source is hosted on GitHub, clone the repository:

```
git clone <repository-url> femm-qt
cd femm-qt
```

4 Building

From the `femm-qt` directory, run the build script:

```
./macos_build.sh
```

This checks that the build tools are present, locates Qt 6 via Homebrew, configures the project with CMake, and builds it with Ninja. The GUI is produced as a macOS application bundle (`build/bin/femm_gui.app`), and the script then assembles everything else into it: the solvers, `femm_headless`, the Lua 4 interpreter, the Python scripting assets, the examples, the language bindings, the manual and the material libraries. `macdeployqt` bundles the Qt frameworks.

The bundle is the *only* place anything runs from on macOS. The loose copies CMake leaves in `build/bin` are deleted once they have been copied in, so that a stray `build/bin/femm_headless` — which would resolve its solvers and material libraries from the wrong directory — cannot be run by accident. The regression suite and the language bindings both reach into the bundle.

The material libraries are a special case, because the material editor writes them and a signed bundle cannot be written to. Read-only copies ship in `Contents/Resources`, and FEMM copies them to `~/Library/Application Support/FEMM/` the first time they are used; that is where your edits go, and they are shared by every copy of FEMM on the machine.

To wipe the build directory and rebuild from scratch, pass `clean`:

```
./macos_build.sh clean
```

If the build fails because CMake cannot find Qt 6, see *Manual configuration* under Troubleshooting.

5 Running

Launch the GUI by double-clicking `femm_gui.app` in Finder, or from Terminal:

```
open build/bin/femm_gui.app
```

Or the headless server (for scripted/IPC use). It lives inside the bundle, beside the GUI:

```
./build/bin/femm_gui.app/Contents/MacOS/femm_headless
```

5.1 How the bindings find FEMM

Usually they need nothing set at all. All six bindings — OctaveFEMM, MathFEMM, SciFEMM, JuliaFEMM, pyFEMM and luaFEMM — look for `femm_gui` and `femm_headless` in four places, in order:

1. `FEMM_DIR`, if it is set;
2. beside the binding's own directory, which covers a source tree — CMake links `femm_gui.app` into `build/bin`, and that is what makes a checkout work with nothing configured;
3. every directory on `PATH`;
4. `/Applications` and `~/Applications`.

The last of those is the macOS case: programs here live in `/Applications` rather than on `PATH`, so a bundle dragged there is found on its own. In each directory the bindings try `femm_gui.app/Contents/MacOS/`*n* as well as the bare name, which is how they reach inside a bundle, and it is why one entry serves both a GUI session (`openfemm()`) and a headless one (`openfemm(1)`) — both executables sit in there together.

Set `FEMM_DIR` only to say *which* copy, when you keep more than one:

```
export FEMM_DIR=/path/to/femm-qt/build/bin
```

Enter it exactly as shown — one line, no spaces around the `=` — and put it in your `~/.zshrc` to make it persistent. Being first in the list, it beats everything else.

The binding files themselves ship twice: read-only inside the bundle at `Contents/Resources/bindings`, and as a plain `~/FEMM/bindings` folder if you dragged `FEMM` out of the disk image. Use the second — Finder will not browse into a `.app`, and Octave and Julia need a real path. Both hold `mfiles` for OctaveFEMM, with `mathfemm`, `scifemm`, `juliafemm`, `pyfemm` and `luafemm` beside it.

6 Setting Up OctaveFEMM

OctaveFEMM drives FEMM from Octave (or MATLAB) over a local socket, and uses Java to make that connection. On a clean macOS machine the setup is:

1. **Install Octave** via Homebrew:

   ```
   brew install octave
   ```

2. **Install a JDK.** OctaveFEMM's socket bridge needs a Java runtime, and Homebrew's Octave does not bundle one. Install the Adoptium Temurin JDK — either download it from `https://adoptium.net` or, more simply, install it with Homebrew:

```
brew install --cask temurin
```

3. **Set `JAVA_HOME`** in your `~/.zshrc`, resolved dynamically with `java_home` so it survives JDK updates:

```
export JAVA_HOME="$(/usr/libexec/java_home)"
```

   Open a new terminal (or run `source ~/.zshrc`) afterward so it takes effect. `FEMM_DIR` is *not* needed: OctaveFEMM finds the application by itself (see "How the bindings find FEMM" above). Set it only if you keep several copies of FEMM and need to choose between them.

4. **Add the m-files to Octave's path.** Start Octave and run, once:

```
addpath('~/FEMM/bindings/mfiles');
savepath;
```

   or, working from a source tree, `addpath('/path/to/femm-qt/octavefemm')`. `savepath` records the directory in your Octave preferences so you do not have to re-add it every session.

   This is a *different* search path from the one in the previous section, and the two are easy to confuse. Octave's path is how Octave finds `openfemm.m`; the search described above is how `openfemm.m` then finds the FEMM application. Only the second is automatic.

To verify, run `openfemm` at the Octave prompt: the FEMM window should appear and connect. Use `openfemm(1)` for a headless session (no window), which talks to `femm_headless` instead.

7 Packaging a Disk Image

To wrap the built bundle in a draggable `.dmg`:

```
packaging/macos_dmg.sh
```

When a Developer ID is configured this script also signs, notarises and staples the image; see *Signing and notarisation* below. To build and package in one step, use `packaging/macos_release.sh` instead, which runs both scripts in the order that matters.

This is a separate step from the build, in the same way that running Inno Setup is separate from compiling on Windows. The bundle has to exist to run and to test; a disk image is only needed to hand FEMM to someone else. The script refuses to run if `femm_gui.app` is missing, incomplete, or stale — the last of those catches a `ninja` that relinked the GUI without reassembling the bundle around it.

The image is written to `packaging/output/` and opens to four things: the application, a symlink to `/Applications`, a `FEMM` folder, and a `Read Me`. Installing is two drags — the app onto `Applications`, and `FEMM` onto the user's `Documents` folder:

```
~/FEMM/examples    models and scripts, every language
~/FEMM/docs        the manual and the per-binding guides
~/FEMM/bindings    the client files for other programs
```

The home folder rather than `~/Documents`, which is where the Linux install puts the same tree, and the difference is deliberate. macOS protects `Documents`, `Desktop` and `Downloads` through TCC: a program that opens a file there under its own steam — as opposed to one you picked in an open panel — needs consent first. An application with a window can ask for it, and `femm_gui` does; a process without one generally just fails with *Operation not permitted*. Worse, an app that fails that way without prompting never appears in **Files & Folders** in System Settings, so there is no obvious way to grant it afterwards — Octave does exactly this. Since driving FEMM from Octave, MATLAB, Python or a shell script is most of what it is for, the home folder, which TCC does not cover, avoids the whole business. It also keeps a directory full of solver output from being synced to iCloud.

It is one folder rather than three at the top level because three would be three things to drag and three things to forget. It has to leave the bundle at all because the bundle is read-only once signed: an example script does `open("model.fem")` and then `mi_saveas("temp.fem")`, both relative to its own directory, and an analysis writes the model back before solving. None of that works inside the application.

The `Read Me` is the macOS stand-in for what the Linux installer prints when it finishes: what to drag where, the `addpath` line for each language, where the material libraries are kept (`~/Library/Application Support/FEMM/`, so they survive an upgrade), and how to uninstall.

That folder is not a duplicate of the copy inside the bundle, and the distinction matters. Example scripts open a model and immediately save it under another name, so that the original is never modified:

```
open("Roters-Ch9Fig6.fem")
mi_saveas("temp.fem")
```

Both paths are relative to the script's own directory, which is where FEMM sets the working directory when it runs a script. Inside the bundle that directory is read-only and the save fails, so drag the `Examples` folder somewhere of your own and work there. The read-only copy inside the bundle is a master, kept so the examples survive throwing the disk image away.

7.1 Signing and notarisation

With nothing configured, `macos_build.sh` applies an ad-hoc signature. That is enough to build, run and test on your own machine, and it is what an ordinary build still does — no Apple account is needed to work on FEMM. It is *not* enough for a disk image handed to anyone else: downloaded from the internet, an ad-hoc signed application is refused by Gatekeeper as coming from an unidentified developer.

Distributing to others needs an Apple Developer ID. Two environment variables turn it on, and the arrangement mirrors `packaging/pack.bat` on Windows — nothing account-specific is committed, and leaving them unset changes nothing:

set	**result**
nothing	ad-hoc; the everyday build
`FEMM_SIGN_ID`	Developer ID, hardened runtime, secure timestamp
+ `FEMM_NOTARY_PROFILE`	notarised and stapled as well

```
export FEMM_SIGN_ID="Developer ID Application: Your Name (TEAMID)"
export FEMM_NOTARY_PROFILE=femm-notary
```

The notary profile is created once, interactively, and then lives in the keychain. It asks for an app-specific password from `appleid.apple.com`, not for your Apple ID password:

```
xcrun notarytool store-credentials femm-notary \
      --apple-id you@example.com --team-id TEAMID
```

With both set, one command cuts a release — the counterpart of `packaging/pack.bat`. It resolves the identity and checks the credentials *before* building anything, so a misconfigured release fails in the first second rather than after a full build:

```
packaging/macos_release.sh
```

Signing is inside-out rather than `codesign --deep`: every nested Mach-O first, the `.app` last, because each signature seals what is beneath it. This is not a stylistic preference. `--deep` is deprecated for signing and a bundle signed that way can fail notarisation; and because `Contents/MacOS` is the code directory, the 54 Python and text files staged there are code objects too, which `--deep` had been signing silently into extended attributes. They are signed explicitly now. Notarisation accepts that layout, which is why `python/` and `pybridge-worker/` can stay where they are.

No entitlements are needed. There is no JIT, and the garbage collector is built with `GC_DISABLE_INCREMENTAL`, so no `SIGSEGV` or Mach exception handler is ever installed — those are the two things that usually force an entitlement under the hardened runtime.

Notarisation is a queue on Apple's side, and 25 minutes per submission is normal rather than a hang; there are two submissions, the application and the image. `FEMM_NOTARY_TIMEOUT` (default `1h`) bounds the wait, and reaching it does not cancel the submission.

8 Troubleshooting

8.1 Manual configuration

The build script wraps the CMake/Ninja steps and assembles the app bundle. The order of the bundling steps matters: `macdeployqt` must run *before* the solvers and data files are copied in (it scans `MacOS/` and aborts on the `.dat` files otherwise), and the bundle must be re-signed afterward. The material libraries are the one thing FEMM writes beside itself, so they are seeded read-only into Contents/Resources/ and copied into the user's Application Support folder on first use; a signed bundle is never written to. To run the steps by hand:

```
mkdir build && cd build
cmake .. -G Ninja \
    -DCMAKE_BUILD_TYPE=Release \
    -DCMAKE_PREFIX_PATH="$(brew --prefix qt@6)"
ninja
APP=bin/femm_gui.app/Contents/MacOS
RES=bin/femm_gui.app/Contents/Resources
# 1. Bundle Qt frameworks FIRST, while the bundle holds only femm_gui.
$(brew --prefix qt@6)/bin/macdeployqt bin/femm_gui.app
# 2. Copy the solvers and the headless engine in afterward:
for s in fkn belasolv hsolv csolv femm_headless; do
    cp bin/$s "$APP/"
done
# 3. Material libraries: read-only SEEDS in Resources/, not MacOS/.
#    FEMM copies them to ~/Library/Application Support/FEMM/ on first
#    use and reads and writes them there, so nothing is ever written
```

```
#    inside the bundle.
mkdir -p "$RES" && cp ../matlibs/*.dat "$RES/"
# 4. Re-seal the bundle with a deep ad-hoc signature:
codesign --force --deep --sign - bin/femm_gui.app
```

The `macos_build.sh` script is the authoritative, commented version of this procedure.

8.2 Qt not found

If CMake reports that it cannot find Qt6, verify the prefix path:

```
brew --prefix qt@6
```

Pass the result to `-DCMAKE_PREFIX_PATH`.

8.3 Permission denied on first run

macOS Gatekeeper may block unsigned applications. If you see a "cannot be opened because the developer cannot be verified" dialog, go to System Settings → Privacy & Security and click "Allow Anyway," or remove the quarantine attribute:

```
xattr -dr com.apple.quarantine build/bin/femm_gui.app
```